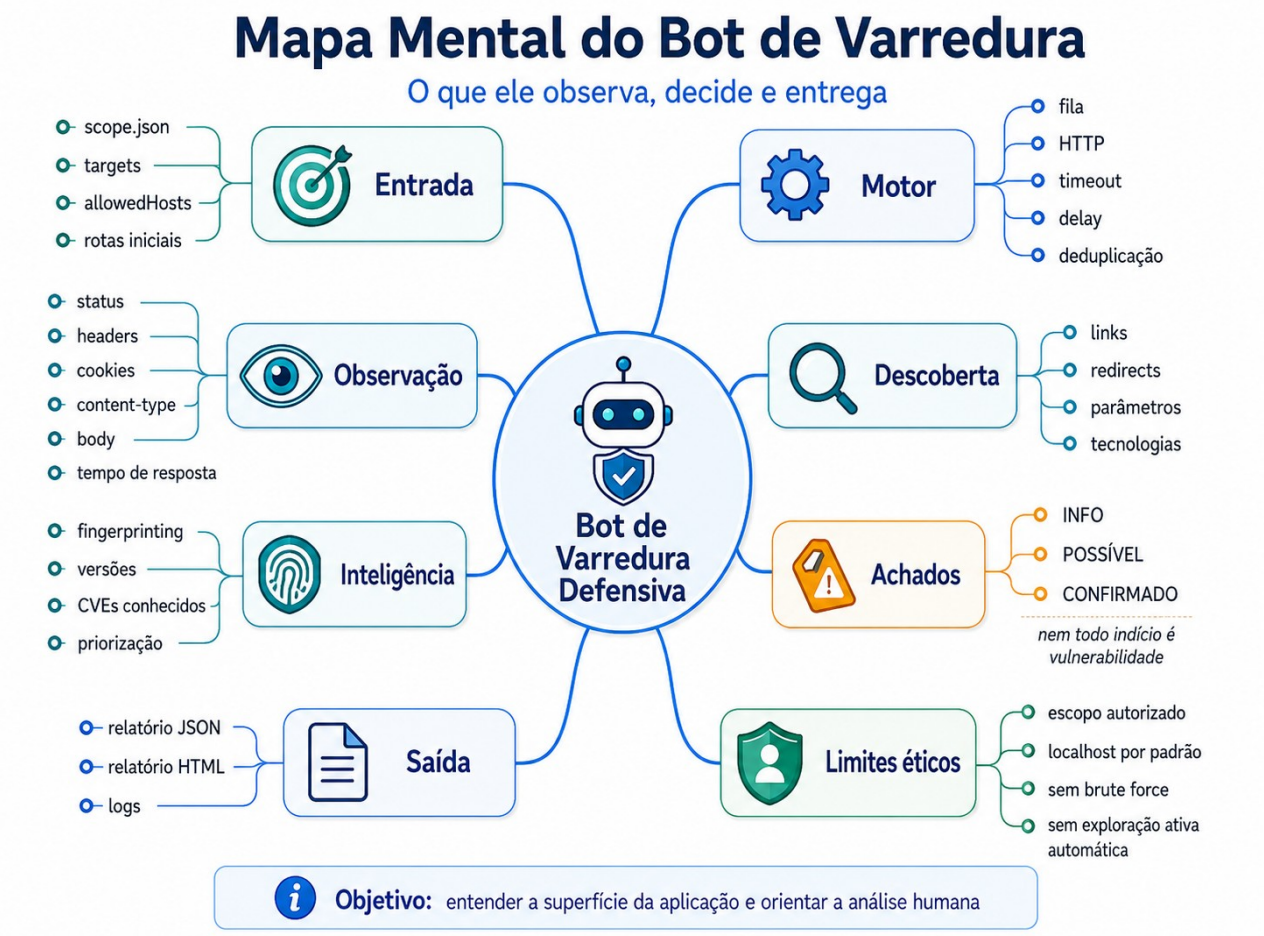
ETHICAL HACKING & APPSEC

Aulas 01 a 40 + Projeto Scanner Defensivo

Material completo de estudo - teoria, exemplos em codigo e equivalentes em Portugol



Mapa geral do scanner defensivo usado como projeto paralelo.

Uso educacional e defensivo. Pratique exploracao e varredura ativa apenas em laboratorio, sistemas proprios ou ambientes com autorizacao explicita.

Edicao consolidada - 2026

Como usar este material

Cada aula contem uma explicacao curta, pontos-chave, um exemplo tecnico e uma versao equivalente em Portugol. O Portugol e didatico: ele traduz a logica, mas nem sempre representa APIs reais de rede, criptografia, banco ou sistema operacional.

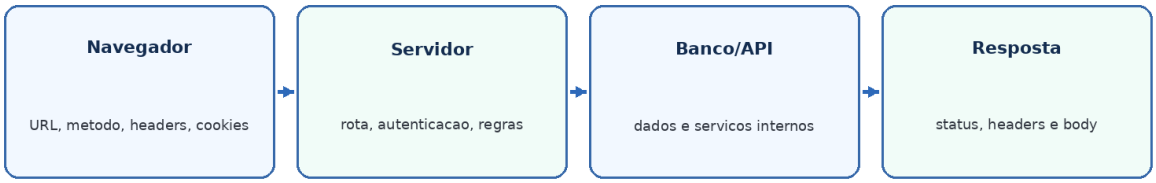
Quando uma aula usa comandos de terminal, Assembly ou filtros de ferramentas, o Portugol mostra a ideia equivalente em vez de tentar substituir a ferramenta real.

Sumario

Aula 1: Fundamentos de Seguranca da Informacao
Aula 2: HTTP, Requests, Responses e Parametros
Aula 3: Autenticacao e Autorizacao
Aula 4: Senhas, Hash, Salt e Bcrypt
Aula 5: Sessoes, Cookies e JWT
Aula 6: IDOR e Broken Access Control
Aula 7: SQL Injection
Aula 8: Cross-Site Scripting - XSS
Aula 9: CSRF
Aula 10: Burp Suite: Proxy, Intercept e Repeater
Aula 11: Seguranca de APIs e Mass Assignment
Aula 12: BOLA / IDOR em APIs
Aula 13: Upload de Arquivos
Aula 14: CORS
Aula 15: SSRF - Server-Side Request Forgery
Aula 16: Business Logic Vulnerabilities
Aula 17: Race Conditions
Aula 18: Privilege Escalation
Aula 19: JWT Avancado, Sessoes e Refresh Tokens
Aula 20: OAuth 2.0, OpenID Connect, State e PKCE
Aula 20.5: Projeto Paralelo - Scanner Defensivo
Aula 21: Redes: IP, Portas, TCP e UDP
Aula 22: Linux para Seguranca
Aula 23: Nmap - Mapeamento de Portas e Servicos
Aula 24: Wireshark - Observando Pacotes
Aula 25: Programas Compilados, CPU e Registradores
Aula 26: Memoria: Stack, Heap e Buffer Overflow
Aula 27: Assembly - Comparacoes e Saltos
Aula 28: Ghidra e Engenharia Reversa
Aula 29: Malware - Conceitos e Analise Segura
Aula 30: Stuxnet e Sistemas Industriais
Aula 31: Metodologia de Pentest
Aula 32: Reconhecimento e Superficie de Ataque
Aula 33: De Pista a Hipotese
Aula 34: Validacao de Impacto
Aula 35: Relatorio Profissional e Reteste
Aula 36: Threat Modeling com STRIDE
Aula 37: Secure Coding
Aula 38: SAST, DAST, SCA e Secret Scanning

Bloco 1 - Fundamentos e seguranca web

Fluxo de uma Aplicacao Web



Seguranca depende principalmente das decisoes do servidor.

Aula 1 - Fundamentos de Seguranca da Informacao

Confidencialidade, Integridade e Disponibilidade formam a triade CIA. A seguranca procura reduzir risco: uma ameaca explora uma vulnerabilidade e pode causar impacto. CVE identifica vulnerabilidades publicas, CVSS ajuda a estimar severidade e CWE classifica tipos de fraqueza.

Pontos-chave

- Diferencie vulnerabilidade, exploit e payload.
- Use menor privilegio e defesa em profundidade.
- Seguranca e gestao de risco, nao promessa de invulnerabilidade.

Exemplo tecnico

Codigo / comando

```
function risco(probabilidade, impacto) {
  return probabilidade * impacto;
}
console.log(risco(3, 4));
```

Portugol - equivalente didatico

```
funcao inteiro risco(inteiro probabilidade, inteiro impacto)
    retorne probabilidade * impacto
fimfuncao

escreva(risco(3, 4))
```

Aula 2 - HTTP, Requests, Responses e Parametros

HTTP organiza a conversa entre cliente e servidor. Uma requisicao tem metodo, caminho, headers e eventualmente body. A resposta possui status, headers e body. Parametros podem aparecer na URL, no corpo ou em headers, mas nunca devem ser considerados confiaveis apenas por terem vindo do frontend.

Pontos-chave

- GET consulta; POST normalmente envia/cria; PATCH altera parcialmente; DELETE remove.
- 200, 301, 400, 401, 403, 404 e 500 sao status comuns.
- Frontend nao substitui validacao no backend.

Exemplo tecnico

Codigo / comando

```
app.get("/produto", (req, res) => {
  const id = req.query.id;
  res.json({ id });
});
```

Portugol - equivalente didatico

```
cadeia id
id <- parametroURL("id")
responderJSON("id", id)
```

Aula 3 - Autenticacao e Autorizacao

Autenticacao responde quem e o usuario; autorizacao responde o que ele pode fazer. Um sistema pode autenticar corretamente e ainda possuir Broken Access Control se nao verificar permissoes em cada recurso.

Pontos-chave

- 401: identidade ausente/invalida em muitos cenarios.
- 403: identidade conhecida, mas sem permissao.
- Autorizacao precisa acontecer no servidor.

Exemplo tecnico

Codigo / comando

```
app.get("/admin", autenticar, (req,res) => {
  if (req.user.role !== "admin") return res.sendStatus(403);
  res.send("Area administrativa");
});
```

Portugol - equivalente didatico

```
se nao autenticado entao
    responder(401)
senao se papel <> "admin" entao
    responder(403)
senao
    escreva("Area administrativa")
fimse
```

Aula 4 - Senhas, Hash, Salt e Bcrypt

Senha nao deve ser armazenada em texto puro. Funcoes de hash de senha como bcrypt usam custo computacional e salt para dificultar ataques contra bancos vazados.

Pontos-chave

- Hash nao e criptografia reversivel.
- Salt evita que senhas iguais produzam o mesmo resultado de forma trivial.
- Use bibliotecas maduras, nao crie algoritmo proprio.

Exemplo tecnico

Codigo / comando

```
const bcrypt = require("bcrypt");
const hash = await bcrypt.hash(password, 12);
const ok = await bcrypt.compare(password, hash);
```

Portugol - equivalente didatico

```
hash <- gerarHashSenha(senha, 12)
ok <- compararSenha(senha, hash)
se ok entao escreva("Senha valida") fimse
```

Aula 5 - Sessoes, Cookies e JWT

Depois do login, o sistema precisa manter estado. Isso pode ocorrer por sessao no servidor identificada por cookie ou por tokens como JWT. Cookies sensiveis devem considerar HttpOnly, Secure e SameSite conforme o contexto.

Pontos-chave

- Cookie e um mecanismo de transporte/armazenamento no cliente.
- JWT assinado nao deve ser apenas decodificado: precisa ser verificado.
- Tokens devem expirar e ser protegidos.

Exemplo tecnico

Codigo / comando

```
const payload = jwt.verify(token, process.env.ACCESS_SECRET);
req.user = payload;
```

Portugol - equivalente didatico

```
dados <- verificarToken(token, SEGREDO)
usuarioAtual <- dados
```

Aula 6 - IDOR e Broken Access Control

IDOR acontece quando um identificador controlado pelo usuario e usado para acessar um objeto sem verificar se aquele usuario pode acessar o objeto. A correcao nao e esconder o ID: e validar propriedade/permissao.

Pontos-chave

- IDs previsiveis nao sao a causa principal.
- O backend deve combinar objeto + identidade autenticada.
- Teste apenas em laboratorio ou escopo autorizado.

Exemplo tecnico

Codigo / comando

```
const pedido = await db.pedidos.findById(req.params.id);
if (pedido.userId !== req.user.id) return res.sendStatus(403);
res.json(pedido);
```

Portugol - equivalente didatico

```
pedido <- buscarPedido(id)
se pedido.usuarioId <> usuarioAtual.id entao
    responder(403)
senao
    responderJSON(pedido)
fimse
```

Aula 7 - SQL Injection

SQL Injection surge quando dados nao confiaveis sao concatenados ao comando SQL. A defesa principal e separar instrucao de dados por queries parametrizadas, alem de validar entradas e usar menor privilegio no banco.

Pontos-chave

- Nunca monte SQL critico por concatenacao de entrada.
- ORM nao elimina automaticamente todo risco.
- Mensagens de erro detalhadas nao devem vazar internamente.

Exemplo tecnico

Codigo / comando

```
const r = await db.query(
  "SELECT * FROM users WHERE email = $1",
  [email]
);
```

Portugol - equivalente didatico

```
consulta <- "SELECT * FROM users WHERE email = ?"
resultado <- executarSQL(consulta, email)
```

Aula 8 - Cross-Site Scripting - XSS

XSS ocorre quando conteudo controlado por usuario vira codigo executavel no navegador de outra pessoa. Context-aware output encoding e frameworks que escapam por padrao ajudam, mas insercoes perigosas de HTML continuam exigindo cuidado.

Pontos-chave

- Stored, Reflected e DOM XSS sao categorias comuns.
- Nao confie apenas em filtros de palavras.
- CSP e camada adicional, nao substitui correcao.

Exemplo tecnico

Codigo / comando

```
function escaparHTML(s) {
  return s.replace(/[&<>"']/g, c => MAP[c]);
}
res.send(`<p>${escaparHTML(nome)}</p>`);
```

Portugol - equivalente didatico

```
nomeSeguro <- escaparHTML(nome)
escreva("<p>" + nomeSeguro + "</p>")
```

Aula 9 - CSRF

CSRF explora o fato de o navegador enviar automaticamente credenciais como cookies para um site. Para operacoes sensiveis, use tokens anti-CSRF quando aplicavel, SameSite e validacao de origem/contexto.

Pontos-chave

- O risco e maior quando autenticacao depende de cookies enviados automaticamente.
- GET nao deve alterar estado sensivel.
- SameSite ajuda, mas arquitetura e contexto importam.

Exemplo tecnico

Codigo / comando

```
if (req.body.csrfToken !== req.session.csrfToken) {
  return res.sendStatus(403);
}
```

Portugol - equivalente didatico

```
se tokenRecebido <> tokenDaSessao entao
    responder(403)
fimse
```

Aula 10 - Burp Suite: Proxy, Intercept e Repeater

Burp atua entre navegador e aplicacao para observar e repetir requisicoes HTTP. Proxy permite interceptar; Repeater ajuda a modificar e reenviar manualmente uma requisicao. O valor didatico e enxergar o que o navegador realmente manda.

Pontos-chave

- Use apenas em ambientes autorizados.
- Compare requisicao original e resposta antes de formular hipoteses.
- Ferramenta nao substitui raciocinio.

Exemplo tecnico

Codigo / comando

```
// Representacao de uma requisicao
const req = { method: "GET", path: "/profile?id=10" };
```

Portugol - equivalente didatico

```
metodo <- "GET"
caminho <- "/profile?id=10"
escreva(metodo + " " + caminho)
```

Aula 11 - Seguranca de APIs e Mass Assignment

APIs recebem objetos estruturados e podem aceitar campos que o usuario nao deveria controlar. Mass Assignment ocorre quando o backend atualiza o modelo diretamente com o body sem selecionar campos permitidos.

Pontos-chave

- Use schemas de entrada.
- Separe DTO de entrada do modelo do banco.
- Campos como role, saldo e status exigem controle explicito.

Exemplo tecnico

Codigo / comando

```
const Schema = z.object({ name: z.string(), email: z.string().email() });
const data = Schema.parse(req.body);
```

Portugol - equivalente didatico

```
se nome = vazio ou emailInvalido(email) entao
    responder(400)
senao
    atualizar(nome, email)
fimse
```

Aula 12 - BOLA / IDOR em APIs

BOLA e Broken Object Level Authorization: a API identifica corretamente o objeto, mas falha ao decidir se o usuario atual pode acessa-lo. E um dos problemas centrais em APIs modernas.

Pontos-chave

- Autorizacao deve ser por objeto.
- UUID nao substitui autorizacao.
- Listagens e exportacoes tambem precisam de filtro por usuario/empresa.

Exemplo tecnico

Codigo / comando

```
const invoice = await db.invoices.findById(req.params.id);
if (invoice.companyId !== req.user.companyId) return res.sendStatus(403);
```

Portugol - equivalente didatico

```
fatura <- buscarFatura(id)
se fatura.empresaId <> usuarioAtual.empresaId entao
    responder(403)
fimse
```

Aula 13 - Upload de Arquivos

Upload pode introduzir arquivos inesperados, conteudo ativo, extensoes enganosas e consumo excessivo. A aplicacao deve limitar tamanho, tipos esperados, nomear arquivos com seguranca e armazenar de forma apropriada.

Pontos-chave

- Nao confie apenas na extensao.
- Defina limite de tamanho e quantidade.
- Evite servir upload executavel como codigo.

Exemplo tecnico

Codigo / comando

```
if (file.size > 5_000_000) return res.status(400).send("Arquivo grande");
if (!ALLOWED_TYPES.includes(file.mimetype)) return res.status(400).send("Tipo invalido");
```

Portugol - equivalente didatico

```
se tamanhoArquivo > 5000000 entao responder(400) fimse
se tipoArquivo nao pertence tiposPermitidos entao responder(400) fimse
```

Aula 14 - CORS

CORS e uma politica do navegador que controla quais origens podem ler respostas entre sites. Nao e autenticacao. Uma configuracao permissiva pode expor dados quando combinada com credenciais e endpoints sensiveis.

Pontos-chave

- Origin deve ser validada conforme necessidade real.
- Nao reflita qualquer Origin cegamente.
- CORS nao protege chamadas servidor-servidor.

Exemplo tecnico

Codigo / comando

```
const permitidas = new Set(["https://app.exemplo.com"]);
if (!permitidas.has(req.headers.origin)) return res.sendStatus(403);
```

Portugol - equivalente didatico

```
se origem nao pertence origensPermitidas entao
    responder(403)
fimse
```

Aula 15 - SSRF - Server-Side Request Forgery

SSRF acontece quando o servidor faz uma requisicao para um destino controlado pelo usuario sem restricao suficiente. No laboratorio, uma rota /preview conseguia acessar um servidor interno. A defesa aplicada foi allowlist de hosts.

Pontos-chave

- O erro estava no fetch(url) com URL nao validada.
- Allowlist e mais segura que blacklist simples.
- Erros de entrada podem usar 400; falhas internas reais podem usar 500.

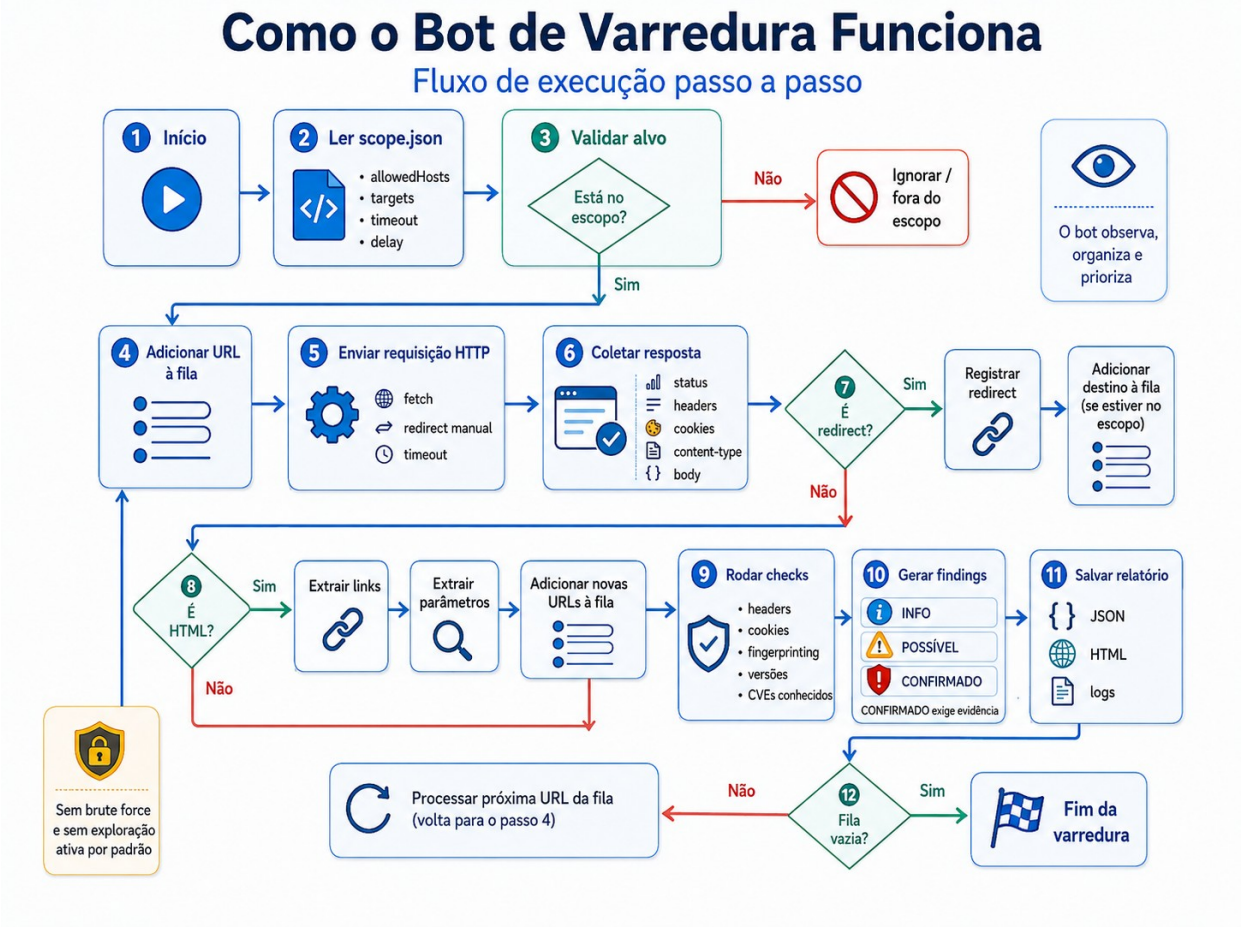
Exemplo tecnico

Codigo / comando

```
const u = new URL(req.query.url);
const permitidos = ["example.com"];
if (!permitidos.includes(u.hostname)) return res.status(400).send("Destino proibido");
const r = await fetch(u);
```

Portugol - equivalente didatico

```
url <- analisarURL(urlRecebida)
se host(url) nao pertence permitidos entao
    responder(400)
senao
    resposta <- requisitar(url)
fimse
```



O scanner posterior reutiliza os mesmos conceitos de requisicao, resposta e escopo.

Bloco 2 - Logica de negocio, tokens e OAuth

Aula 16 - Business Logic Vulnerabilities

Falhas de logica de negocio surgem quando o fluxo tecnicamente funciona, mas as regras permitem estados ou combinacoes que nao deveriam existir: quantidade negativa, cupom reutilizado, preco enviado pelo cliente ou etapa pulada.

Pontos-chave

- Teste zero, negativo, repeticao e ordem das etapas.
- Valores economicos devem ser calculados no servidor.
- Estado atual deve autorizar a proxima transicao.

Exemplo tecnico

Codigo / comando

```
if (!Number.isInteger(qtd) || qtd < 1) return res.status(400).send("Quantidade invalida");
const produto = await db.produtos.findById(produtoId);
const total = produto.preco * qtd;
```

Portugol - equivalente didatico

```
se quantidade < 1 entao responder(400) fimse
produto <- buscarProduto(produtoId)
total <- produto.preco * quantidade
```

Aula 17 - Race Conditions

Uma race condition ocorre quando operacoes corretas isoladamente produzem estado incorreto quando executadas ao mesmo tempo. Saldos, estoque, cupons e reservas sao exemplos tipicos.

Pontos-chave

- Check-then-act sem exclusao pode falhar.
- Transacoes e bloqueios do banco ajudam.
- Idempotencia e restricoes unicas tambem sao importantes.

Exemplo tecnico

Codigo / comando

```
await db.query("BEGIN");
const saldo = await db.query("SELECT saldo FROM contas WHERE id=$1 FOR UPDATE", [id]);
// validar e atualizar
await db.query("COMMIT");
```

Portugol - equivalente didatico

```
iniciarTransacao()
bloquearConta(id)
saldo <- lerSaldo(id)
se saldo >= valor entao atualizarSaldo(id, saldo-valor) fimse
confirmarTransacao()
```

Aula 18 - Privilege Escalation

Escalada vertical ocorre quando um usuario consegue privilegios maiores; horizontal quando acessa recursos de outro usuario no mesmo nivel. O cliente nunca deve decidir seu proprio papel.

Pontos-chave

- Rotas administrativas precisam de verificacao explicita.
- Self-service nao deve aceitar role.
- Permissoes precisam ser aplicadas em cada acao sensivel.

Exemplo tecnico

Codigo / comando

```
const data = { name: req.body.name, email: req.body.email };
await db.users.update(req.user.id, data);
```

Portugol - equivalente didatico

```
dados.nome <- nomeRecebido
dados.email <- emailRecebido
// papel nao e aceito aqui
atualizarUsuario(usuarioAtual.id, dados)
```

Aula 19 - JWT Avancado, Sessoes e Refresh Tokens

Access tokens devem ser curtos; refresh tokens podem durar mais e precisam ser protegidos e revogaveis. jwt.decode apenas le; jwt.verify valida assinatura e claims conforme configuracao.

Pontos-chave

- Separe segredos/chaves e finalidades.
- Valide exp, issuer e audience quando aplicavel.
- Logout forte pode exigir revogacao/rotacao do refresh token.

Exemplo tecnico

Codigo / comando

```
const access = jwt.sign({ sub: user.id }, ACCESS_SECRET, { expiresIn: "15m" });
const payload = jwt.verify(access, ACCESS_SECRET);
```

Portugol - equivalente didatico

```
token <- assinarToken(usuario.id, 15_minutos)
dados <- verificarToken(token, SEGREDO_ACESSO)
```

Aula 20 - OAuth 2.0, OpenID Connect, State e PKCE

OAuth e principalmente delegacao de autorizacao; OpenID Connect adiciona identidade. No Authorization Code Flow, a aplicacao recebe um codigo e troca por tokens. state liga resposta ao fluxo iniciado; PKCE protege contra reutilizacao de codigo interceptado.

Pontos-chave

- Redirect URI deve ser estrita.
- state deve ser imprevisivel e verificado.
- OIDC exige validacao correta de issuer, audience e assinatura.

Exemplo tecnico

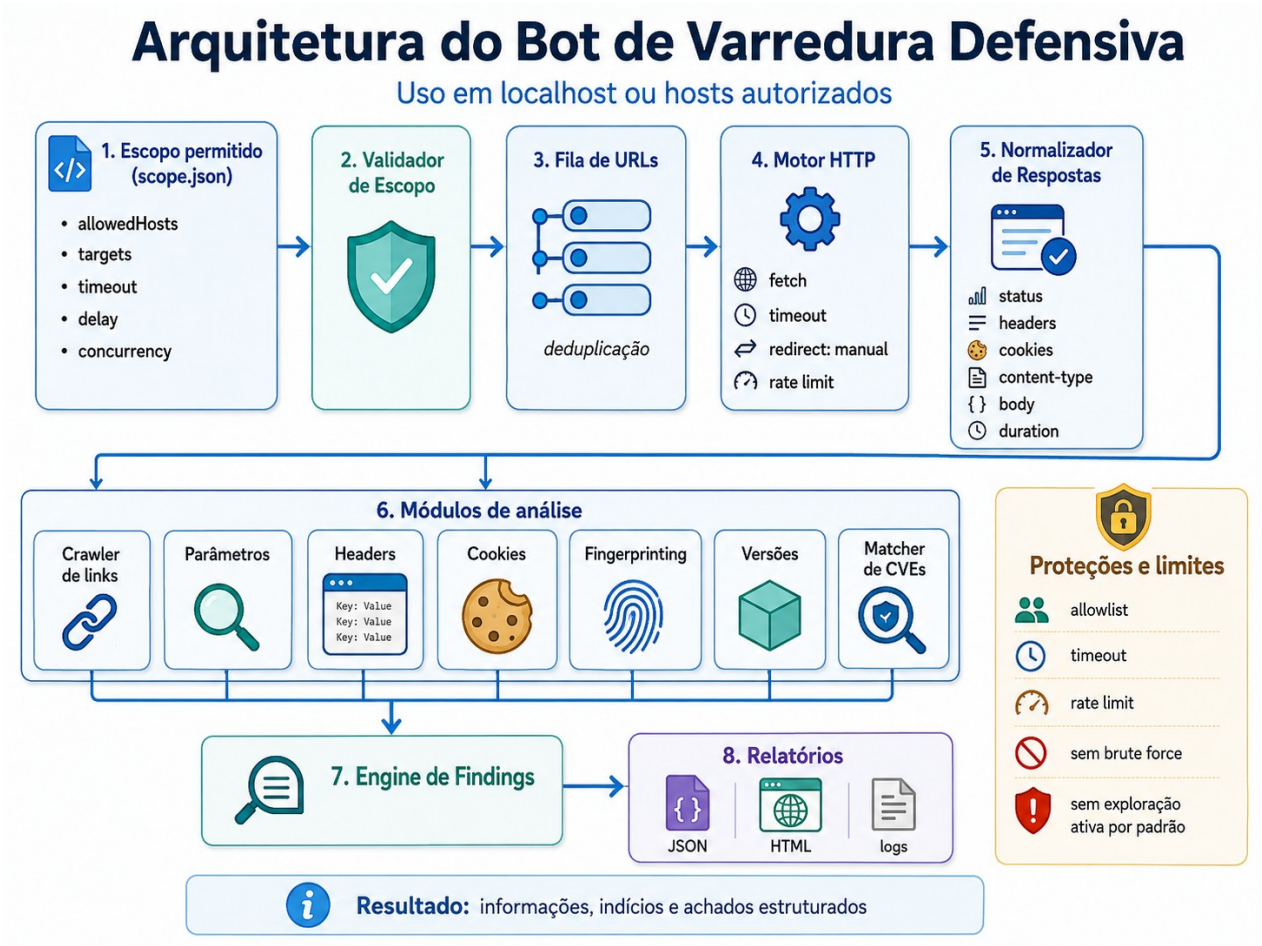
Codigo / comando

```
req.session.oauthState = crypto.randomUUID();
// callback
if (req.query.state !== req.session.oauthState) return res.sendStatus(400);
```

Portugol - equivalente didatico

```
estadoSessao <- gerarUUID()
se estadoRecebido <> estadoSessao entao
    responder(400)
fimse
```

Projeto paralelo - Scanner Defensivo



Projeto paralelo - Scanner Defensivo

Aula 20.5 - Projeto Paralelo - Scanner Defensivo

O scanner evoluiu de uma lista fixa de rotas para uma arquitetura modular: scope, fila, HTTP, crawler, parametros, headers, cookies, fingerprinting, versoes, correlacao local de CVEs e relatorios. O escopo e sempre validado antes de qualquer requisicao.

Pontos-chave

- Allowlist define hosts autorizados.
- Timeout, delay e maxPages limitam comportamento.
- Correlacao de versao gera POSSIVEL, nao CONFIRMADO automaticamente.

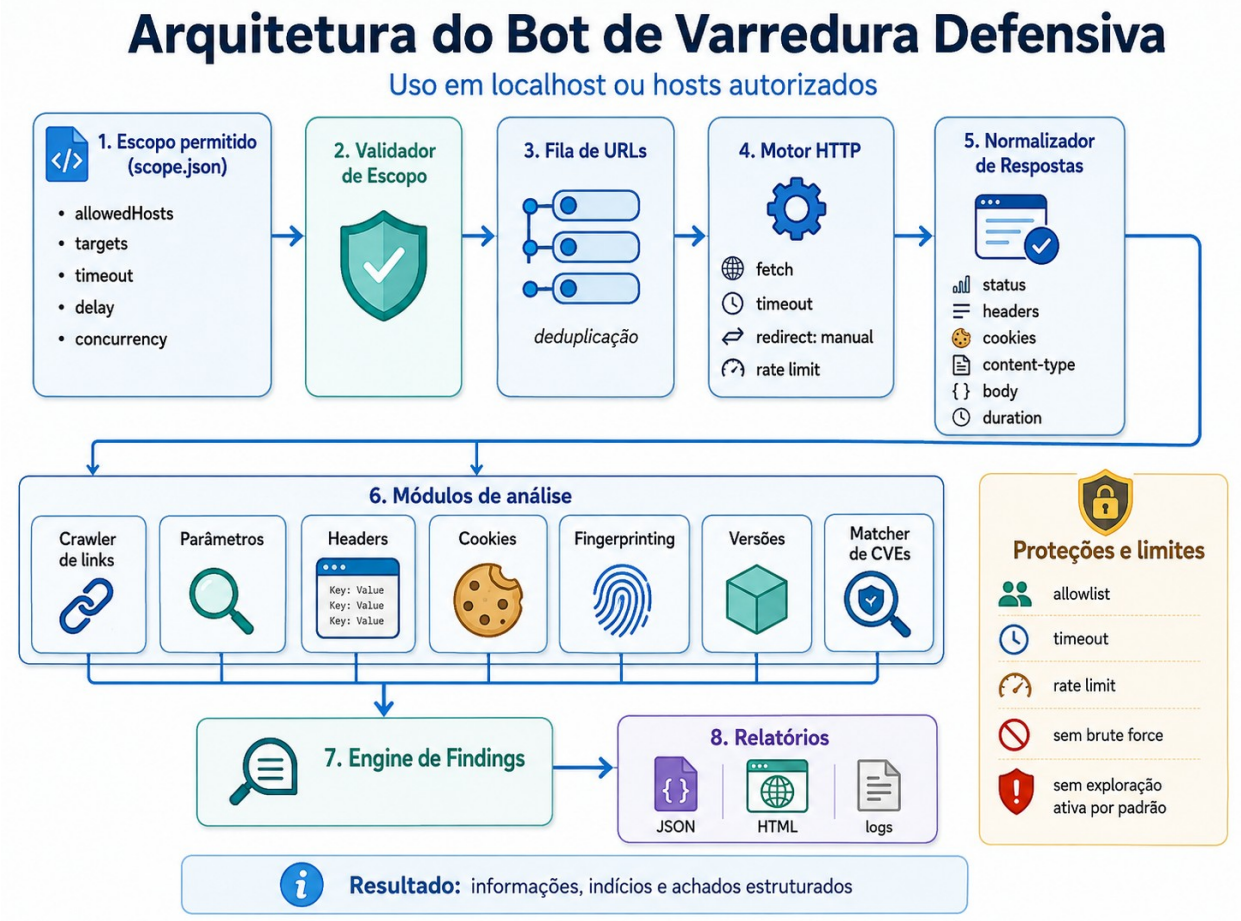
Exemplo tecnico

Codigo / comando

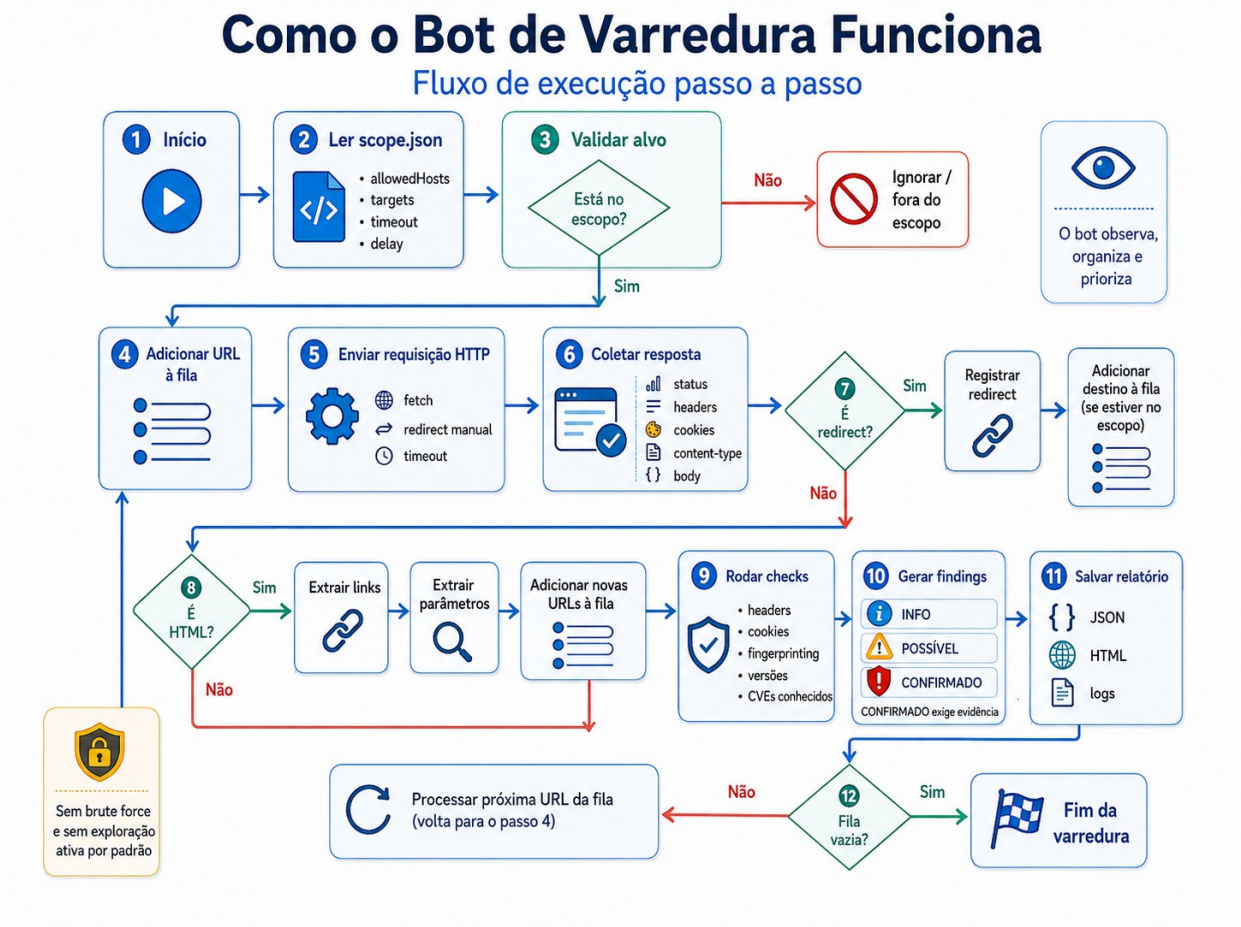
```
const url = validarAlvo(target);
queue.add(url.toString());
while (queue.hasNext()) {
  const result = await request(queue.next());
  // analisar resposta e gerar findings
}
```

Portugol - equivalente didatico

```
alvo <- validarAlvo(alvoRecebido)
adicionarFila(alvo)
enquanto filaNaoVazia() faca
    resposta <- requisitar(proximaURL())
    analisar(resposta)
fimenquanto
```



Arquitetura modular do Scanner Defensivo.



Fluxo de execucao do Scanner Defensivo.

Bloco 3 - Redes e Linux

Da URL ao Pacote de Rede



Nmap observa portas; Wireshark observa trafego; Burp observa HTTP.

Bloco 3 - Redes e Linux

Aula 21 - Redes: IP, Portas, TCP e UDP

IP identifica uma interface na rede; porta identifica um servico naquela maquina. TCP cria uma conexao confiavel orientada a fluxo; UDP envia datagramas sem a mesma garantia de entrega. localhost normalmente aponta para 127.0.0.1.

Pontos-chave

- 80 HTTP, 443 HTTPS, 22 SSH e 53 DNS sao exemplos comuns.
- LISTENING significa que um processo aguarda conexoes.
- Uma mesma maquina pode ter varios servicos em portas diferentes.

Exemplo tecnico

Codigo / comando

```
const net = require("net");
const s = net.connect(3000, "127.0.0.1", () => {
  console.log("Porta acessivel"); s.end();
});
```

Portugol - equivalente didatico

```
se conectarTCP("127.0.0.1", 3000) entao
    escreva("Porta acessivel")
fimse
```

Aula 22 - Linux para Seguranca

Linux e frequente em servidores e ferramentas de seguranca. Comandos como pwd, ls, cd, cat, ps, grep e ss ajudam a navegar, ler arquivos, localizar processos e observar portas.

Pontos-chave

- Pipe | envia a saida de um comando para outro.
- Permissoes r/w/x significam leitura, escrita e execucao.
- No WSL e possivel praticar Linux dentro do Windows.

Exemplo tecnico

Codigo / comando

```
# Shell
ps aux | grep node
ss -tulpn
ls -la
```

Portugol - equivalente didatico

```
// Portugol conceitual
processos <- listarProcessos()
mostrar(filtrar(processos, "node"))
mostrar(listarPortas())
```

Aula 23 - Nmap - Mapeamento de Portas e Servicos

Nmap identifica portas abertas, fechadas ou filtradas e pode tentar reconhecer servicos. Em laboratorio, ele permite confirmar os servidores Node nas portas 3000 e 4000.

Pontos-chave

- open: servico aceitando conexao.
- closed: host respondeu, mas nada escuta ali.
- filtered: filtragem impede conclusao clara.

Exemplo tecnico

Codigo / comando

```
# Somente laboratorio/local
nmap -p 3000,4000 127.0.0.1
nmap -sV -p 3000,4000 127.0.0.1
```

Portugol - equivalente didatico

```
portas <- [3000, 4000]
para cada porta em portas faca
    escreva(testarPorta("127.0.0.1", porta))
fimpara
```

Aula 24 - Wireshark - Observando Pacotes

Wireshark captura e interpreta pacotes. Uma requisicao web passa por camadas como Ethernet, IP, TCP e HTTP/TLS. Filtros ajudam a isolar o trafego relevante.

Pontos-chave

- tcp.port == 3000 filtra a porta do laboratorio.
- http mostra trafego HTTP reconhecido.
- HTTPS cifra o conteudo da aplicacao em transito.

Exemplo tecnico

Codigo / comando

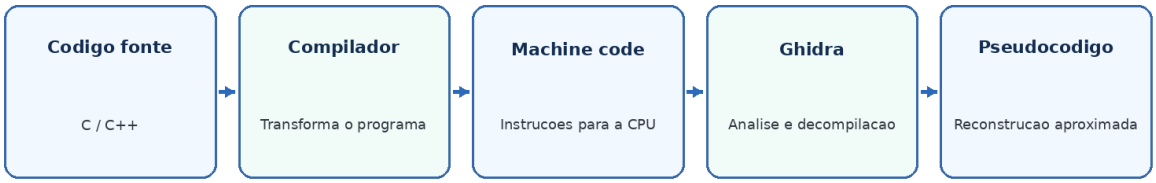
```
// Filtros Wireshark
tcp.port == 3000
http
dns
```

Portugol - equivalente didatico

```
se pacote.porta = 3000 entao
    mostrar(pacote)
fimse
```

Bloco 4 - Binarios e engenharia reversa

Do Codigo a Engenharia Reversa



O decompilador reconstrui logica, mas nao recupera perfeitamente o fonte original.

Bloco 4 - Binarios e engenharia reversa

Aula 25 - Programas Compilados, CPU e Registradores

C e compilado em instrucoes de maquina. Assembly representa essas instrucoes de forma legivel. Registradores sao pequenas areas muito rapidas dentro da CPU e participam de calculos, chamadas e controle de fluxo.

Pontos-chave

- RAX/EAX e frequentemente usado para resultados.
- RSP aponta para a stack.
- RIP indica a proxima instrucao a executar em x86-64.

Exemplo tecnico

Codigo / comando

```
// C
int x = 5;
int y = 3;
int z = x + y;
// Assembly conceitual: mov eax,5; add eax,3
```

Portugol - equivalente didatico

```
inteiro x <- 5
inteiro y <- 3
inteiro z <- x + y
escreva(z)
```

Aula 26 - Memoria: Stack, Heap e Buffer Overflow

Stack guarda chamadas e variaveis locais; heap guarda alocacoes dinamicas. Linguagens como C permitem controle direto de memoria, por isso erros de limite podem corromper memoria. Buffer overflow e estudado apenas em laboratorio controlado.

Pontos-chave

- Stack segue comportamento de pilha.
- malloc aloca no heap e free libera.
- Sempre valide tamanhos e prefira APIs seguras.

Exemplo tecnico

Codigo / comando

```
char nome[8];
// forma segura: limitar a copia ao tamanho do buffer
snprintf(nome, sizeof(nome), "%s", entrada);
```

Portugol - equivalente didatico

```
cadeia nome
leia(entrada)
se comprimento(entrada) <= 7 entao
    nome <- entrada
senao
    escreva("Entrada grande demais")
fimse
```

Aula 27 - Assembly - Comparacoes e Saltos

Assembly mostra como if, loops e chamadas viram comparacoes e saltos. cmp compara valores e instrucoes como je/jne alteram o fluxo conforme flags da CPU.

Pontos-chave

- mov move/carrega valores.
- add/sub realizam aritmetica.
- cmp + jump implementam decisoes.

Exemplo tecnico

Codigo / comando

```
; Assembly conceitual
cmp eax, 10
je igual
mov ebx, 0
jmp fim
igual:
mov ebx, 1
fim:
```

Portugol - equivalente didatico

```
se x = 10 entao
    resultado <- 1
senao
    resultado <- 0
fimse
```

Aula 28 - Ghidra e Engenharia Reversa

Ghidra analisa binarios e tenta reconstruir funcoes, strings, referencias e pseudocodigo. Decompilacao e aproximada: nomes de variaveis e estruturas originais podem ter sido perdidos.

Pontos-chave

- Analise estatica nao executa o binario.
- Strings e imports oferecem pistas.
- Compare pseudocodigo com assembly quando algo parecer estranho.

Exemplo tecnico

Codigo / comando

```
// Fonte hipotetico
int validar(int n) {
  return n == 1234;
}
// Ghidra pode reconstruir logica semelhante.
```

Portugol - equivalente didatico

```
funcao logico validar(inteiro n)
    retorne n = 1234
fimfuncao
```

Aula 29 - Malware - Conceitos e Analise Segura

Malware e software com comportamento malicioso. Analise defensiva comeca por hash, strings, metadados, imports e engenharia reversa antes de qualquer execucao. Ransomware, spyware, worm e trojan descrevem familias/comportamentos diferentes.

Pontos-chave

- Nao execute arquivo suspeito no computador comum.
- Analise estatica primeiro.
- Sandbox isolada e usada apenas quando a analise dinamica e necessaria.

Exemplo tecnico

Codigo / comando

```
const crypto = require("crypto");
const hash = crypto.createHash("sha256").update(buffer).digest("hex");
console.log(hash);
```

Portugol - equivalente didatico

```
hash <- calcularSHA256(arquivo)
escreva(hash)
```

Aula 30 - Stuxnet e Sistemas Industriais

Stuxnet e um caso historico de malware voltado a ambiente industrial. O ponto didatico e entender a cadeia digital-fisica: Windows, software industrial, PLCs, processo e telemetria. Aqui estudamos arquitetura e impacto, nao codigo operacional de malware.

Pontos-chave

- PLC controla processos fisicos.
- Ataques ICS podem alterar processo e observacao.
- Seguranca industrial exige conhecimento de TI e engenharia.

Exemplo tecnico

Codigo / comando

```
// Modelo defensivo de monitoramento
if (Math.abs(sensor - referencia) > limite) {
  gerarAlerta();
}
```

Portugol - equivalente didatico

```
se valorAbsoluto(sensor - referencia) > limite entao
    gerarAlerta()
fimse
```

Bloco 5 - Metodologia de pentest

Ciclo Profissional de Pentest



Depois da correcao vem o reteste.

Aula 31 - Metodologia de Pentest

Pentest profissional comeca por autorizacao e escopo, segue por reconhecimento, hipoteses, validacao minima, avaliacao de impacto, relatorio e reteste. Ferramenta e apenas parte do processo.

Pontos-chave

- Defina o que esta dentro e fora do escopo.
- Pare quando a evidencia minima for suficiente.
- Documente horarios, endpoints e resultados.

Exemplo tecnico

Codigo / comando

```
function noEscopo(url, hosts) {
  return hosts.includes(new URL(url).hostname);
}
```

Portugol - equivalente didatico

```
funcao logico noEscopo(cadeia url, lista hosts)
    retorne host(url) pertence hosts
fimfuncao
```

Aula 32 - Reconhecimento e Superficie de Ataque

Superficie de ataque e o conjunto de sistemas, rotas, APIs, parametros, portas e tecnologias expostos e autorizados para avaliacao. Reconhecimento organiza esse inventario antes de testes mais profundos.

Pontos-chave

- Mapeie hosts e aplicacoes.
- Catalogue rotas e parametros.
- Diferencie ativos proprios de terceiros.

Exemplo tecnico

Codigo / comando

```
const u = new URL("https://app.exemplo.com/produto?id=10");
console.log(u.hostname, u.pathname, [...u.searchParams.keys()]);
```

Portugol - equivalente didatico

```
host <- obterHost(url)
rota <- obterRota(url)
parametros <- obterParametros(url)
mostrar(host, rota, parametros)
```

Aula 33 - De Pista a Hipotese

Um scanner deve diferenciar observacao de vulnerabilidade. Um parametro id e uma pista; uma versao aparente pode sugerir um CVE; nenhuma das duas confirma impacto sozinha.

Pontos-chave

- INFO: observacao.
- POSSIVEL: merece validacao.
- CONFIRMADO: exige evidencia suficiente.

Exemplo tecnico

Codigo / comando

```
function classificar(evidencia) {
  if (evidencia.confirmada) return "CONFIRMADO";
  if (evidencia.indicio) return "POSSIVEL";
  return "INFO";
}
```

Portugol - equivalente didatico

```
se evidencia.confirmada entao
    retorne "CONFIRMADO"
senao se evidencia.indicio entao
    retorne "POSSIVEL"
senao
    retorne "INFO"
fimse
```

Aula 34 - Validacao de Impacto

Severidade depende de contexto: dados afetados, autenticacao necessaria, alcance, capacidade de leitura/alteracao e facilidade. O objetivo e provar risco sem causar dano desnecessario.

Pontos-chave

- Compare impacto e probabilidade.
- Dados de terceiros elevam impacto.
- Nao destrua ou extraia mais do que o necessario.

Exemplo tecnico

Codigo / comando

```
function score(impacto, probabilidade) {
  return Math.min(10, impacto * probabilidade);
}
```

Portugol - equivalente didatico

```
pontuacao <- impacto * probabilidade
se pontuacao > 10 entao pontuacao <- 10 fimse
```

Aula 35 - Relatorio Profissional e Reteste

Um finding profissional inclui titulo, severidade, endpoint, descricao, pre-requisitos, reproducao, resultado observado, esperado, impacto, evidencia e recomendacao. Depois da correcao, o reteste determina se foi resolvido.

Pontos-chave

- Evite linguagem sensacionalista.
- Remova dados sensiveis desnecessarios da evidencia.
- Registre status RESOLVIDO ou AINDA PRESENTE no reteste.

Exemplo tecnico

Codigo / comando

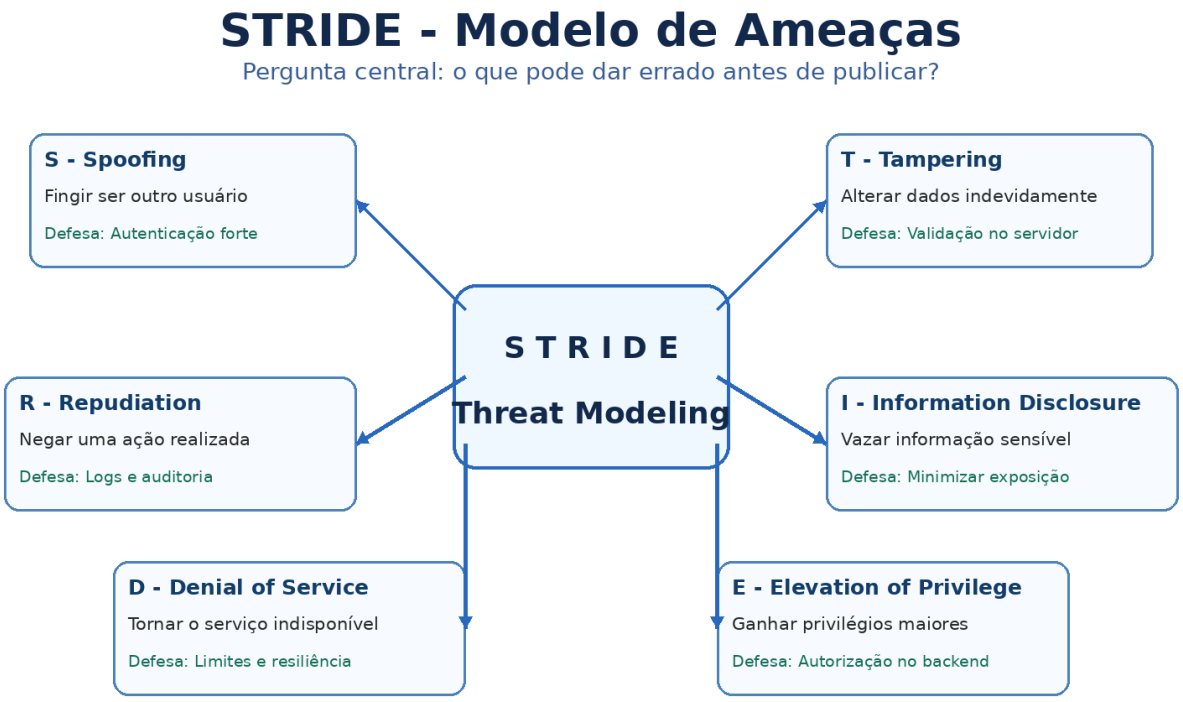
```
const finding = {
  title: "Controle de acesso inadequado",
  severity: "HIGH",
  endpoint: "/profile",
  recommendation: "Validar autorizacao no servidor"
};
```

Portugol - equivalente didatico

```
registro finding
    cadeia titulo
    cadeia severidade
    cadeia endpoint
    cadeia recomendacao
fimregistro
```

Um bom relatorio separa observacao, hipotese, evidencia, impacto e recomendacao.

Bloco 6 - Desenvolvimento seguro e projeto final



Bloco 6 - Desenvolvimento seguro e projeto final

Aula 36 - Threat Modeling com STRIDE

Threat Modeling pergunta o que pode dar errado antes da publicacao. STRIDE cobre Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service e Elevation of Privilege.

Pontos-chave

- Modele fluxos e fronteiras de confianca.
- Relacione cada ameaca a controles.
- Revise quando arquitetura mudar.

Exemplo tecnico

Codigo / comando

```
const stride = ["Spoofing","Tampering","Repudiation","Information Disclosure","Denial of
Service","Elevation of Privilege"];
for (const threat of stride) console.log(threat);
```

Portugol - equivalente didatico

```
ameacas <- ["Spoofing", "Tampering", "Repudiation", "Information Disclosure", "Denial of Service",
"Elevation of Privilege"]
para cada ameaca em ameacas faca escreva(ameaca) fimpara
```

Aula 37 - Secure Coding

Secure Coding incorpora validacao, autenticacao, autorizacao, queries parametrizadas, protecao de segredos, logs adequados e regras de negocio no desenvolvimento diario.

Pontos-chave

- Cliente pede; servidor valida e decide.
- Segredos ficam fora do codigo.
- Falhas internas sao logadas sem vazar detalhes ao cliente.

Exemplo tecnico

Codigo / comando

```
const ProductSchema = z.object({ productId: z.number().int().positive(), quantity:
z.number().int().min(1).max(100) });
const data = ProductSchema.parse(req.body);
```

Portugol - equivalente didatico

```
se produtoId <= 0 ou quantidade < 1 ou quantidade > 100 entao
    responder(400)
senao
    processar(produtoId, quantidade)
fimse
```

Aula 38 - SAST, DAST, SCA e Secret Scanning

SAST analisa codigo; DAST analisa aplicacao rodando; SCA analisa dependencias; Secret Scanning procura chaves e tokens. As camadas se complementam.

Pontos-chave

- npm audit e um exemplo de SCA no ecossistema Node.
- Scanner proprio esta mais proximo de DAST/recon.
- Segredo exposto deve ser revogado, nao apenas apagado.

Exemplo tecnico

Codigo / comando

```
const checks = ["SAST", "SCA", "Secret Scanning", "DAST"];
for (const check of checks) executar(check);
```

Portugol - equivalente didatico

```
verificacoes <- ["SAST", "SCA", "Secret Scanning", "DAST"]
para cada item em verificacoes faca executar(item) fimpara
```

Aula 39 - DevSecOps

DevSecOps coloca verificacoes de seguranca no ciclo de entrega: codigo, testes, SAST, SCA, segredo, build, deploy, DAST e monitoramento. Falhas criticas podem bloquear publicacao.

Pontos-chave

- Detectar cedo reduz custo.
- Pipeline deve ser reproduzivel.
- Monitoramento continua depois do deploy.

Exemplo tecnico

Codigo / comando

```
async function pipeline() {
  await testes(); await sast(); await sca();
  await build(); await deploy(); await dast();
}
```

Portugol - equivalente didatico

```
procedimento pipeline()
    testes()
    sast()
    sca()
    build()
    deploy()
    dast()
fimprocedimento
```

Aula 40 - Projeto Final - Pentest Completo de Laboratorio

O fechamento do primeiro ciclo e um pentest de laboratorio propositalmente vulneravel. O trabalho combina escopo, reconhecimento, scanner, testes manuais, evidencia, correcao e reteste.

Pontos-chave

- Priorize raciocinio antes de payloads.
- Produza relatorio inicial e final.
- O objetivo e demonstrar encontrar e corrigir problemas.

Exemplo tecnico

Codigo / comando

```
const report = {
  target: "localhost",
  findings: [],
  retest: []
};
fs.writeFileSync("report.json", JSON.stringify(report, null, 2));
```

Portugol - equivalente didatico

```
relatorio.alvo <- "localhost"
relatorio.achados <- listaVazia()
relatorio.reteste <- listaVazia()
salvarJSON(relatorio)
```

Entrega final sugerida: relatorio inicial + correcoes + reteste + relatorio final.

Apendice A - Glossario rapido

Termo	Definicao
AAA	Authentication, Authorization, Accounting/Auditing.
API	Interface para comunicacao entre sistemas.
BOLA	Broken Object Level Authorization.
CVE	Identificador publico de vulnerabilidade.
CVSS	Sistema de pontuacao de severidade.
CWE	Catalogo de tipos de fraqueza.
DAST	Teste dinamico contra aplicacao em execucao.
Fingerprinting	Identificacao provavel de tecnologia por pistas.
Finding	Achado documentado de uma avaliacao.
IDOR	Acesso indevido a objeto por falha de autorizacao.
OIDC	Camada de identidade sobre OAuth 2.0.
PKCE	Protecao adicional do Authorization Code Flow.
SAST	Analise estatica do codigo.
SCA	Analise de componentes/dependencias.
SSRF	Servidor faz requisicao a destino controlado indevidamente.
XSS	Conteudo nao confiavel executado como script no navegador.

Apendice B - Checklist de estudo pratico

- Refazer em PC as aulas 16-20: logica, race condition, privilegios, JWT e OAuth.
- Rodar Linux/WSL, Nmap e Wireshark no laboratorio local.
- Compilar um pequeno programa C e abrir no Ghidra.
- Executar SAST/SCA/DAST em projeto proprio.
- Concluir Scanner Defensivo V4 e revisar o relatorio HTML/JSON.
- Fazer o pentest final do laboratorio, corrigir e retestar.

Proximo bloco planejado: Web Security avancada, seguida por binarios/reverse engineering, malware analysis, Red Team, cloud, mobile e IoT/ICS.